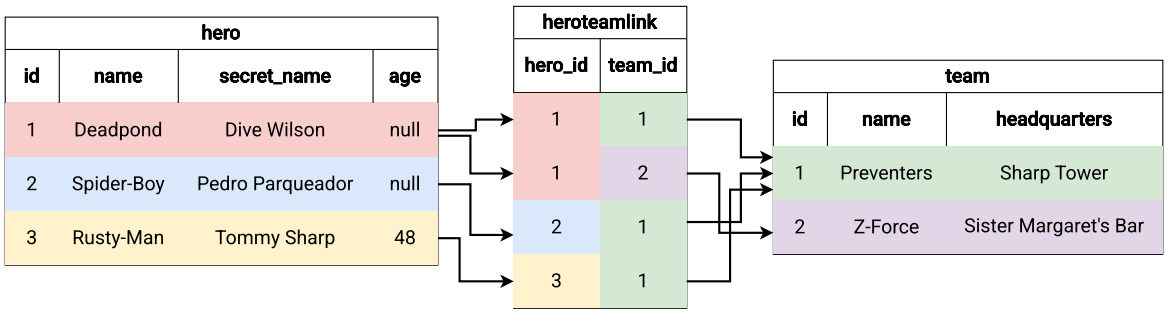
Create Models with a Many-to-Many Link

We'll now support **many-to-many** relationships using a **link table** like this:



Let's start by defining the class models, including the **link table** model.

Link Table Model

As we want to support a **many-to-many** relationship, now we need a **link table** to connect them.

We can create it just as any other **SQLModel**:

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

This is a **SQLModel** class model table like any other.

It has two fields, `team_id` and `hero_id`.

They are both **foreign keys** to their respective tables. We'll create those models in a second, but you already know how that works.

And **both fields are primary keys**. We hadn't used this before. 🤓

Team Model

Let's see the `Team` model, it's almost identical as before, but with a little change:

Python 3.10+

```python
# Code above omitted 👆

class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

The **relationship attribute** `heroes` is still a list of heroes, annotated as `list["Hero"]`. Again, we use `"Hero"` in quotes because we haven't declared that class yet by this point in the code (but as you know, editors and **SQLModel** understand that).

We use the same `Relationship()` function.

We use `back_populates="teams"`. Before we referenced an attribute `team`, but as now we can have many, we'll rename it to `teams` when creating the `Hero` model.

And here's the important part to allow the **many-to-many** relationship, we use `link_model=HeroTeamLink`. That's it. ✨

Hero Model

Let's see the other side, here's the `Hero` model:

Python 3.10+

```python
# Code above omitted 👆

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

We **removed** the previous `team_id` field (column) because now the relationship is done via the link table. 🔥

The relationship attribute is now named **`teams`** instead of `team`, as now we support multiple teams.

It no longer has a type of `Team | None` but a list of teams, the type is now declared as **`list[Team]`**.

We are using the **`Relationship()`** here too.

We still have **`back_populates="heroes"`** as before.

And now we have a **`link_model=HeroTeamLink`**. ✨

Create the Tables

The same as before, we will have the rest of the code to create the **engine**, and a function to create all the tables `create_db_and_tables()`.

Python 3.10+

```python
# Code above omitted 👆

sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

And as in previous examples, we will add that function to a function `main()`, and we will call that `main()` function in the main block:

Python 3.10+

```python
# Code above omitted 👆

def main():
    create_db_and_tables()

# Code here omitted 👈

if __name__ == "__main__":
    main()
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Run the Code

If you run the code in the command line, it would output:

```bash
```

Recap

We can support **many-to-many** relationships between tables by declaring a link table.

We can create it the same way as with other **SQLModel** classes, and then use it in the `link_model` parameter to `Relationship()`.

Now let's work with data using these models in the next chapters. 🤓